CS50 SQL · FINAL PROJECT

My Spotify Wrapped 2026

Building my own year-in-music recap with SQLite

Francisco Mendez, PhD

PURPOSE & SCOPE

One listener, one year, every stream

Listening history collected from the Spotify developer API, imported from CSV into spotify_wrapped_2026.db to answer the questions Spotify Wrapped answers.

4,214	1,910	551	Jan–Sep
streams	tracks	artists	Jan 8 – Sep 18, 2026

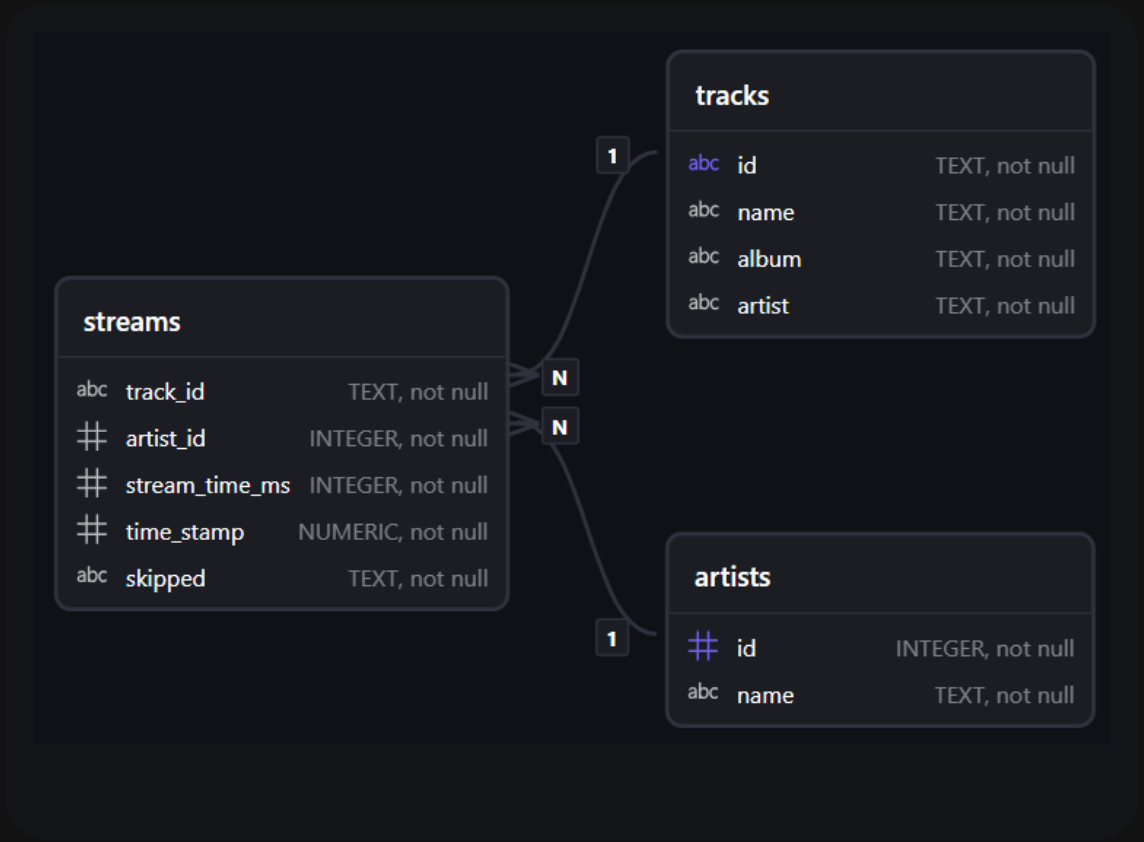
queries.sql

```
-- Insert the data into the respective tables
.import --csv --skip 1 ./artists.csv artists
.import --csv --skip 1 ./tracks.csv tracks
.import --csv --skip 1 ./streams.csv streams
```

In scope: tracks, artists, and every play with its duration, timestamp, and skip flag
Out of scope: other users, genres, audio features, playlists, podcasts, and other years

REPRESENTATION

A small star schema: streams at the center



schema.sql

```sql
-- Create the streams table
CREATE TABLE streams (
    "track_id" TEXT NOT NULL,
    "artist_id" INTEGER NOT NULL,
    "stream_time_ms" INTEGER NOT NULL DEFAULT 0,
    "time_stamp" NUMERIC NOT NULL,
    "skipped" TEXT NOT NULL
        CHECK("skipped" IN ('True','False')),
    FOREIGN KEY("track_id") REFERENCES tracks("id"),
    FOREIGN KEY("artist_id") REFERENCES artists("id")
);
```

tracks 1 —< N **streams** >— 1 **artists** · track id = Spotify URI · duration in ms · ISO 8601 UTC timestamps

OPTIMIZATIONS

Indexes for joins and filters, a view for daily totals

schema.sql · indexes

```
-- Index the foreign key columns in the streams table
CREATE INDEX "stream_track_index" ON "streams"("track_id");
CREATE INDEX "stream_artist_index" ON "streams"("artist_id");

-- Index the skipped column for faster filtering
CREATE INDEX "stream_skipped_index" ON "streams"("skipped");
```

schema.sql · view

```
CREATE VIEW "daily_streaming_time" AS
SELECT
    strftime('%Y-%m-%d', time_stamp) AS Day,
    (SUM(stream_time_ms) / 1000) / 60 AS
total_stream_minutes
FROM streams
GROUP BY Day;
```

Faster joins

track_id and artist_id indexes back every JOIN and GROUP BY in the top songs / artists queries.

Skip filtering

Supports WHERE skipped = 'True'. Low selectivity (31% skips), but cheap to keep for a bulk-loaded table.

Reusable logic

The view keeps daily totals in one place for the top days and daily average questions.

QUERIES · TOTAL LISTENING TIME

13,123 minutes streamed in 2026

13,123
total streaming minutes

218.7
hours listened

9.1
days of continuous audio

CALCULATION BREAKDOWN

- Spotify API returns duration in milliseconds (**stream_time_ms**)
- Integer division: **(SUM(...) / 1000) / 60** yields whole minutes
- Covers all **4,214 streams** recorded between Jan 8 and Sep 18

queries.sql

```sql
-- How many total minutes did I stream music in 2026?
SELECT
    -- Calculate the total listening time in minutes
    (SUM(stream_time_ms) / 1000) / 60 AS total_stream_minutes
FROM
    streams
;
```

QUERIES · TOP LISTENED

Top songs and artists by total minutes

queries.sql · top songs

```sql
-- What are the top 10 songs I streamed in 2026?
SELECT
    t.name AS Song,
    a.name AS Artist,
    (SUM(stream_time_ms) / 1000) / 60 AS total_stream_minutes
FROM streams AS s
    JOIN tracks AS t ON s.track_id = t.id
    JOIN artists AS a ON s.artist_id = a.id
GROUP BY t.id
ORDER BY total_stream_minutes DESC
LIMIT 10;
```

queries.sql · top artists

```sql
-- What are the top 10 artists I streamed in 2026?
SELECT
    a.name AS Artist,
    (SUM(stream_time_ms) / 1000) / 60 AS total_stream_minutes
FROM streams AS s
    JOIN artists AS a
    ON s.artist_id = a.id
GROUP BY a.id
ORDER BY total_stream_minutes DESC
LIMIT 10;
```

TOP 5 SONGS BY MINUTES

1	**Spit Out The Bone** · Metallica	130 min
2	**Damage, Inc.** · Metallica	103 min
3	**Sweet** · Lana Del Rey	84 min
4	**Leper Messiah** · Metallica	84 min
5	**Battery** · Metallica	81 min

TOP 5 ARTISTS BY MINUTES

1	**Metallica**	3,875 min
2	**Ghost**	862 min
3	**Rammstein**	697 min
4	**Megadeth**	514 min
5	**Disturbed**	417 min

QUERIES · TOP SKIPPED

Filtering on skipped = 'True'

queries.sql · skipped songs

```sql
-- What are the top 10 songs I skipped?
SELECT
    t.name AS Song,
    a.name AS Artist,
    COUNT(s.skipped) AS "Total Skips"
FROM streams AS s
    JOIN tracks AS t ON s.track_id = t.id
    JOIN artists AS a ON s.artist_id = a.id
WHERE s.skipped = 'True'
GROUP BY t.id
ORDER BY "Total Skips" DESC
LIMIT 10;
```

queries.sql · skipped artists

```sql
-- What are the top 10 artists I skipped?
SELECT
    a.name AS Artist,
    COUNT(s.skipped) AS "Total Skips"
FROM streams AS s
    JOIN artists AS a
    ON s.artist_id = a.id
WHERE s.skipped = 'True'
GROUP BY a.id
ORDER BY "Total Skips" DESC
LIMIT 10;
```

TOP 5 SKIPPED SONGS

1	**Here Comes Revenge** · Metallica	6 skips
2	**The Ecstasy Of Gold** · Ennio Morricone	5 skips
3	**Post Mortem / Raining Blood** · Havok	5 skips
4	**Masked Ball - 1999 Extended Mix** · Jocelyn Pook	5 skips
5	**Duel of the Fates** · John Williams	5 skips

TOP 5 SKIPPED ARTISTS

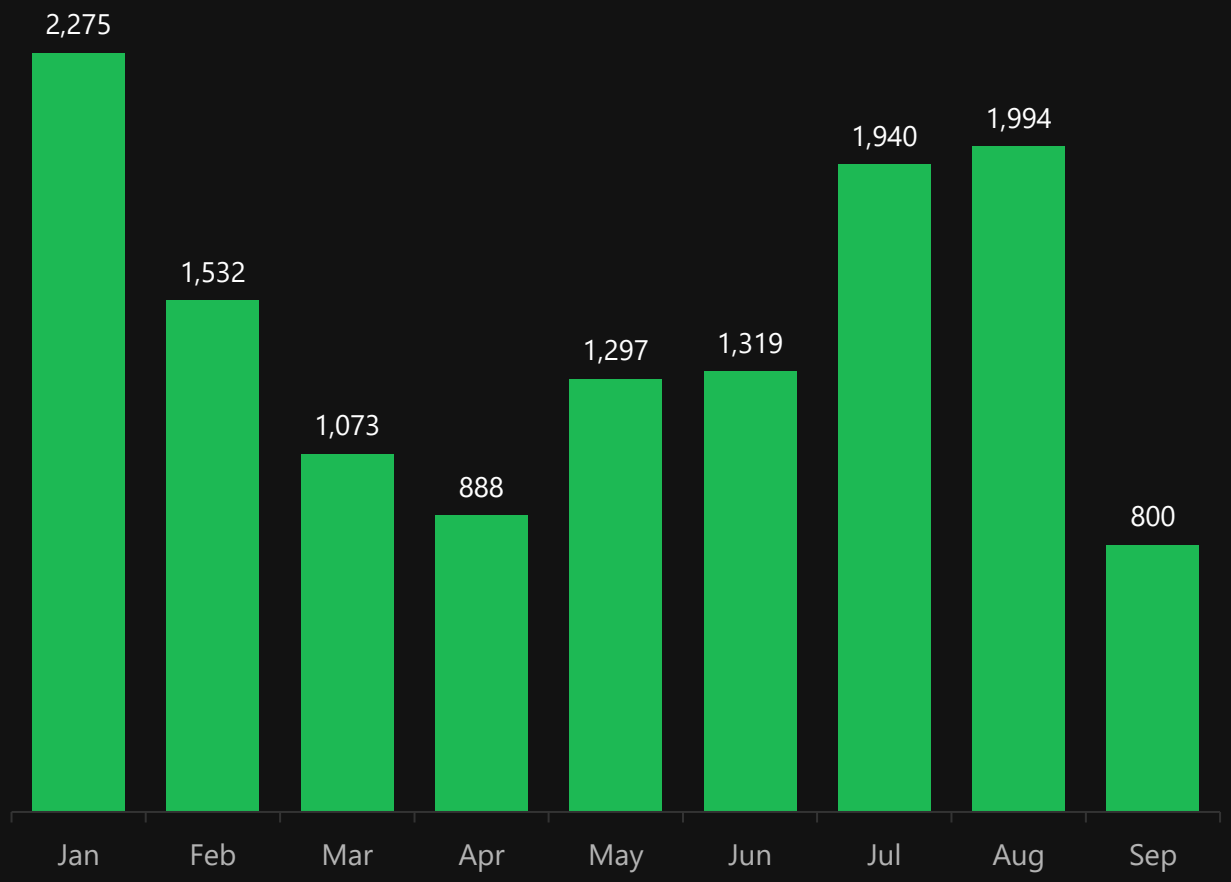
1	**Metallica**	175 skips
2	**Megadeth**	71 skips
3	**Rammstein**	66 skips
4	**Ghost**	66 skips
5	**Disturbed**	47 skips

THE SKIP PARADOX

Metallica leads both: 3,875 min streamed and 175 skips.
Heavy rotation gives artists far more opportunities to be skipped.

QUERIES · MONTHLY LISTENING

Listening distribution across 2026



queries.sql · monthly query

```sql
-- What is my total streaming time by month?
SELECT
    CASE strftime('%m', s.time_stamp)
        WHEN '01' THEN 'January'
        WHEN '02' THEN 'February'
        WHEN '03' THEN 'March'
        WHEN '04' THEN 'April'
        WHEN '05' THEN 'May'
        WHEN '06' THEN 'June'
        WHEN '07' THEN 'July'
        WHEN '08' THEN 'August'
        WHEN '09' THEN 'September'
    END AS Month,
    (SUM(stream_time_ms) / 1000) / 60
        AS total_stream_minutes
FROM
    streams AS s
GROUP BY
    Month
ORDER BY
    strftime('%m', s.time_stamp)
;
```

QUERIES · DAILY ANALYSIS

Top 10 streaming days and daily average

queries.sql · with CTE

```sql
-- Top 10 days WITH a Common Table Expression (CTE)
WITH daily_streams AS (
    SELECT
        strftime('%Y-%m-%d', time_stamp) AS Day,
        (SUM(stream_time_ms) / 1000) / 60 AS total_stream_minutes
    FROM streams
    GROUP BY Day
)
SELECT Day, total_stream_minutes
FROM daily_streams
ORDER BY total_stream_minutes DESC
LIMIT 10;
```

queries.sql · without CTE (using View)

```sql
-- Alternatively, use the daily_streaming_time View
SELECT
    Day,
    total_stream_minutes
FROM
    daily_streaming_time
ORDER BY
    total_stream_minutes DESC
LIMIT 10;
```

queries.sql · daily average

```sql
-- Daily Average listening time
SELECT
    ROUND(AVG(total_stream_minutes), 0)
        AS daily_average_minutes
FROM (
    SELECT
        strftime('%Y-%m-%d', time_stamp) AS Day,
        (SUM(stream_time_ms) / 1000) / 60
            AS total_stream_minutes
    FROM streams
    GROUP BY Day
);
```

2026-05-09

peak day · 428 min

75 min

daily average (active days)

Why a View? Encapsulates the daily group-by logic so any downstream report gets clean daily totals without duplicating CTE code.